The definitions as they stand

What the theory defines, what it leaves undefined, and what I would change, 27 October 2026

The definitions fall in three layers. The oracle layer and the measurement layer are settled, and every proven result rests on them. The price layer is defined piecewise, with each game defining its own attacker and budget. The attacker you want, a learner with a budget in learning units, has no definition anywhere. That gap is where the doubt belongs, and the oracle is not the problem. Sections 1 to 3 state what exists, section 4 states what is missing, and section 5 gives my pushbacks with three decisions for you. A definition carries no epistemic label. Every claim about one does.

1. The oracle layer

Definition 1 (learning function). A measurable map $\mathsf{L}(x; \rho, \xi)$. The training input x names the learner, the starting model, the dataset and the hypothesis class. The seed ρ has an atomless law μ, and the hardware randomness ξ has law π.

Definition 2 (learning oracle). It answers three calls, and every name it returns carries a label, open or eval.

- $\mathsf{Train}(x, s)$. With $s = \perp$ it draws a fresh $\rho \sim \mu$ and stores it under a new name. With s a seed name it reuses that seed. In both cases it draws a fresh $\xi \sim \pi$, stores $\mathsf{L}(x; \rho, \xi)$ and returns its name.
- $\mathsf{Eval}(h, u)$ returns the evaluation of the stored model h at u.
- $\mathsf{Open}(h)$ returns the stored object.

Definition 3 (procedure and adversary). A procedure is a dataflow graph whose learning nodes call Train and whose evaluation nodes call Eval. Each node value reaches the coalition through the plain channel, the query channel (an eval name, a query set U and an allowance Q), or no channel. An adversary is an oracle algorithm with a budget (q_T, q_E) of its own calls. It either distinguishes two hypotheses H_0, H_1, scored by the hockey-stick divergence E_γ between its view laws, or steers a node's value into a target set.

Definition 4 (seed-access modes). In the named mode a caller uses Definition 2 as stated. In the drawn mode it may call only $\mathsf{Train}(x, \perp)$, the seed gets no name the caller holds, and Open on a seed is refused. The bottom oracle gives the adversary the named mode and no honest seed. The top oracle also gives it the seed name of every honest learning node. A cost meter $\mathsf{cost}(x) = (\mathsf{work}(x), \mathsf{size}(x))$ prices one run, and a caller's oracle cost is the sum over its Train calls.

Three proven facts settle this layer.

- The oracle is realizable. Proven (`thm-perfect-substitution`). Against an adversary that samples L once per call, the real procedure, run with fresh randomness, gives the bottom oracle's joint

law of honest values and transcript. Every bound proven under the bottom oracle therefore holds for the real system.
- An unbounded attacker cannot see the oracle. Proven (booklet 6.4(a)). The best advantage satisfies $\epsilon_{\mathsf{L}}(q_T, q_E) = \epsilon_{\mathsf{L}}(0, q_E)$, because an attacker that knows L simulates its own Train calls.
- An experimenter sees it through the seed name. Proven (booklet 3.8 and 6.4(c)). Named-seed runs certify the coupled divergence of Definition 6 for every L. Through a deterministic channel with atomless view laws, no drawn-mode certificate gives an upper band below one.

This answers your question of whether the oracle's presence is distinguishable from its absence. For an attacker under the bottom oracle it is not, and that is what lets the bounds transfer to the real system. For a certifier it is, and the seed name is the entire difference. The oracle's content is the seed name and the count of Train calls.

2. The measurement layer

Definition 5 (identity coupling and twin). For training inputs x, x', the identity coupling is the law of $(M, M') = (\mathsf{L}(x; \rho, \xi), \mathsf{L}(x'; \rho, \xi'))$ with one seed $\rho \sim \mu$ and independent $\xi, \xi' \sim \pi$. When x' differs from x in one record, M' is the twin of M.

Definition 6 (coupled divergence and coupled tail). For a channel C from models to views and $\gamma \geq 1$, the coupled divergence is $\bar{E}_\gamma = \mathbb{E}_{(M,M')} E_\gamma\big(C(M) \,\|\, C(M')\big)$ over the identity coupling. For a norm on contributions, the coupled tail is $\epsilon(\Delta) = \Pr[\|M - M'\| > \Delta]$, taken over every round in the isolated-client setting of Definition 8.

Definition 7 (certifier). An experimenter with at most N Train calls that outputs $\hat{t}$ for a target $\theta(\mathsf{L}) \in [0, 1]$. It is sound at level α over a class $\mathfrak{L}$ when $\Pr[\theta(\mathsf{L}) \leq \hat{t}] \geq 1 - \alpha$ for every $\mathsf{L} \in \mathfrak{L}$. It follows the drawn-seed discipline when it never passes or opens a seed, and the coupled discipline otherwise.

Definition 8 (isolated client). For R rounds, client j trains with drawn seeds, no other node reads its dataset, and its round-t output reaches the coalition only through $y_t = O_t + f_t(c_t, W_t) + Z_t$ with $Z_t \sim \mathcal{N}(0, \sigma^2 I_d)$.

Results 1 to 9 of the reading of 26 October rest on these four. I see nothing to change here.

3. The price layer

Definition 9 (uniform security profile). Fix an instance class $\mathfrak{J}$ and a budget $\beta = (m, N)$ of m reference samples and N Train calls. The informed optimum $\mathrm{opt}(J)$ is the best success of any finite-budget adversary allowed to depend on the instance J. The uniform regret is

$$g(\beta) = \inf_A \sup_{J \in \mathfrak{J}} \big[\mathrm{opt}(J) - \mathrm{succ}_J(A)\big],$$

the infimum taken over adversaries of budget β fixed before J. The attained price at accuracy a is the set of minimal budgets at which one adversary has gap at most a on every instance. The threshold price of a criterion is the set of minimal budgets at which one adversary meets it on every instance. Both sets are finite by Dickson's lemma.

The class is there for a reason. Proven (remark of `def-uniform-security-profile`). On one instance the regret is zero, since an informed adversary simulates every finite-budget one. An unbounded attacker's price exists only as the price of not knowing the instance.

Definition 10 (reconstruction games on $\mathcal{P}_H$). The class $\mathcal{P}_H$, for a closed subgroup $H \le O(d)$, holds split protocols with an H-equivariant client, an H-invariant initialization law and an H-equivariant first-layer update, whose server never sees first-layer weights, their gradients, or records. Two games use it.

- In the sample game the attacker sees one run on N records and runs in time T. Its fine success at scale ϵ is the chance that some symmetry $\kappa \in K$ of the target g puts every output within ϵ of $\kappa g(x_i)$, minus the same chance for fresh records.
- In the population SQ game the client is frozen, and the attacker makes q queries of tolerance τ to the law of (representation, side data). Its weak success is the correlation of its output map with κg.

A seed-known variant also gives the initialization. The price of the seed is the pair of budget sets at which the seed-known and the seed-blind attackers reach a success level.

Definition 11 (mapping). A mapping between an attack task and a hard problem from the literature names the procedure with its cost meter, the task with its success and baseline, the hard problem, the direction of the reduction with the training runs it simulates, the price on each side, the regime, the exponent ratio, the carrier of the hardness, and an experiment. A mapping whose only hard instance is a key-dependent change made for security does not count.

Three attacker classes appear, and each is defined inside the game that uses it.

Attacker class	Defined in	Budget
unbounded	Definition 9	samples m, Train calls N
statistical query	Definition 10, population game	queries q, tolerance τ, or VSTAT(t)
polynomial time	Definition 10, sample game	time T, under a named assumption

4. What is not defined

The learning attacker. Your target is an attacker that is itself a learner, such as a VAE or a shadow model, with a budget in learning units. No definition says this. The SQ class stands in for it, through one borrowed fact. Mini-batch SGD with batch b and gradient precision τ, with $b\tau^2$ above a logarithmic factor, is simulated by statistical queries (AKMSS21 Theorem 1c, borrowed, read in full by a subagent). The fact is used, but no object in the theory names the attacker it covers.

The unit. Four units appear: reference samples, Train calls, queries with a tolerance, and time. The cost meter of Definition 4 exists, and only one remark uses it. The two results that compare attacker and trainer in one unit compare growth exponents. The extraction result compares d^2 with d. The same-units result gives the ratio $\Theta(1 + n^{1-2a})$ of the attacker's price to the trainer's.

The attack and defence statements. Neither needs its own game, but the theory has never written them down. For an attacker class $\mathcal{A}$, a task, a success level s and an instance class $\mathfrak{F}$, let $\text{price}_{\mathcal{A}}(s)$ be the set of minimal budgets at which some $A \in \mathcal{A}$ reaches s on every $J \in \mathfrak{F}$. An attack result puts a budget in its upward closure. A defence result puts every budget below a bound outside it. Your decision of two games, never mixed, fixes $\mathcal{A}$ once per statement, either SQ or polynomial time.

5. Pushbacks

1. Freeze the oracle layer. It is realizable and invisible to the unbounded attacker, and both facts are proven. Its whole content is the seed name and the count of runs. The choices still open in it, such as which mode a game grants, are already parameters that each result names. More rounds on these definitions would only choose those parameters again.

2. Define the gradient attacker and prove that it embeds in the SQ class. A proposal follows.

   Definition (gradient attacker). An (n, T, b, τ) gradient attacker fixes a parametric family $\{f_\theta\}$ with p parameters, a loss ℓ and an initialization law. It draws n pairs from the joint law of (what it sees, the target), runs T steps of mini-batch gradient descent with batch b and gradient precision τ, and outputs f_{θ_T}.

   The proposition to prove is the embedding into SQ, with the query count and tolerance read off AKMSS21 Theorem 1c. With it, results 20 and 21 of 26 October become statements about learning attackers. Without it, "learning attacker" is only a phrase. Your direction has a cost here. Result 17 shows lattice reduction beating every SQ tester, on discrete hidden labels in exact arithmetic, so a defence against gradient attackers says nothing against an attacker who runs LLL. The theory can then prove that practical attacks fail, but not that every efficient attack fails.

3. One price definition for every class. Section 4's $\text{price}_{\mathcal{A}}(s)$ is the threshold price of Definition 9 with the class made explicit, and it covers Definition 10's prices as cases. The informed optimum and the regret are needed for the unbounded class alone, where the price would otherwise vanish. I would keep them there and drop them elsewhere.

4. Use the exponent as the unit. At fixed success, a price that grows as $\beta(d) \asymp d^e$ has unit e, and the comparison with the trainer is the ratio $e_{\text{att}}/e_{\text{tr}}$. Samples, runs and queries then become comparable. The cost meter stays as bookkeeping for experiments. This is a choice, not a theorem, and it loses constants, which matter to a reviewer reading a table of attack costs.

5. Split Definition 11. Its mathematical content is a reduction R from the hard problem P to the task, with a map $\beta_P = \phi(\beta_{\text{task}})$ on budgets and a map on success. The regime, the carrier and the experiment are a reporting checklist. The carrier rule is better read as an admissibility rule than as a field.

Three decisions are yours.

1. Freeze Definitions 1 to 4?
2. Adopt the gradient attacker as the learning attacker class, with the SQ embedding as the next proof?
3. Adopt the exponent as the unit of comparison?